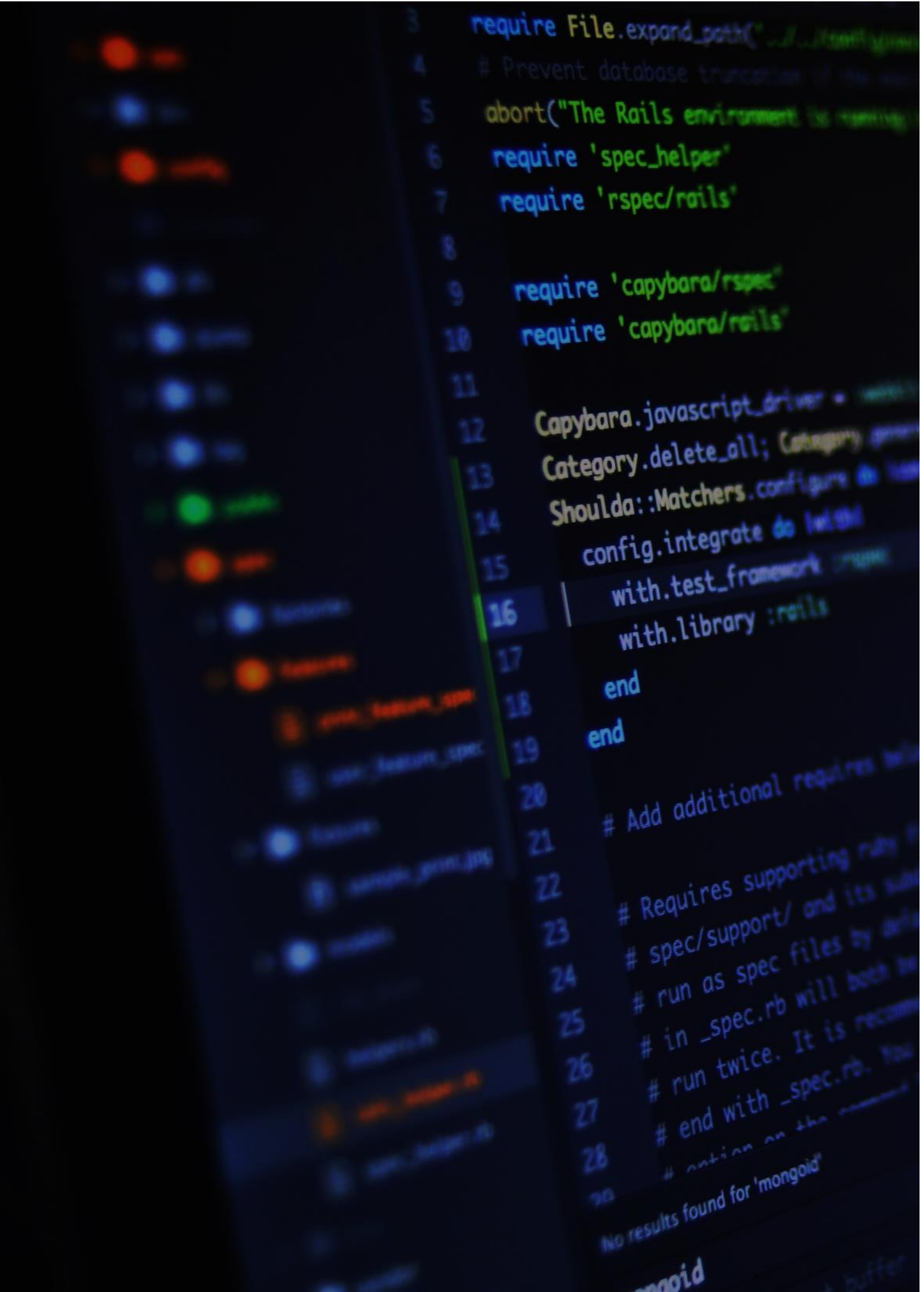


LECTURE 7

continue SQL constrains - Relations

WEB DEVELOPMENT 2

DR. ASHRAF ABDELAZIZ TAHA

MySQL Constraints

- SQL constraints are used to specify rules for data in a table.

Create Constraints

Constraints can be specified when the table is created with the CREATE TABLE statement, or after the table is created with the ALTER TABLE statement.

```
CREATE TABLE table_name (
    column1 datatype constraint,
    column2 datatype constraint,
    column3 datatype constraint,
    ....
);
```

MySQL Constraints

The following constraints are commonly used in SQL:

- `NOT NULL` - Ensures that a column cannot have a NULL value
- `UNIQUE` - Ensures that all values in a column are different
- `PRIMARY KEY` - A combination of a `NOT NULL` and `UNIQUE`.
  Uniquely identifies each row in a table
- `FOREIGN KEY` - Prevents actions that would destroy links between tables

MySQL NOT NULL Constraint

NOT NULL on CREATE TABLE

```
CREATE TABLE Persons (
    ID int NOT NULL,
    LastName varchar (255) NOT NULL,
    FirstName varchar (255) NOT NULL,
    Age int
);
```

NOT NULL on ALTER TABLE

```
ALTER TABLE Persons
MODIFY Age int NOT NULL;
```

MySQL UNIQUE Constraint

UNIQUE on CREATE TABLE

```
CREATE TABLE Persons (
    ID int NOT NULL,
    LastName varchar(255) NOT NULL,
    FirstName varchar(255),
    Age int,
    UNIQUE (ID)
);
```

```
CREATE TABLE Persons (
    ID int NOT NULL,
    LastName varchar(255) NOT NULL,
    FirstName varchar(255),
    Age int,
    CONSTRAINT UC_Person UNIQUE (ID,LastName)
);
```

UNIQUE Constraint on ALTER TABLE

```
ALTER TABLE Persons
ADD CONSTRAINT UC_Person UNIQUE (ID,LastName);
```

DROP a UNIQUE Constraint

- To drop a UNIQUE constraint, use the following SQL:

```
ALTER TABLE Persons
DROP CONSTRAINT UC_Person;
```

MySQL PRIMARY KEY Constraint

- The PRIMARY KEY constraint uniquely identifies each record in a table.
- Primary keys must contain UNIQUE values, and cannot contain NULL values.
- A table can have only ONE primary key; and in the table, this primary key can consist of single or multiple columns (fields).

MySQL PRIMARY KEY Constraint

- The following SQL creates a PRIMARY KEY on the "ID" column when the "Persons" table is created:

```
CREATE TABLE Persons (
   ID int NOT NULL,
   LastName varchar(255) NOT NULL,
   FirstName varchar(255),
   Age int,
   PRIMARY KEY (ID)
);
```

```
CREATE TABLE Persons (
   ID int PRIMARY KEY ,
   LastName varchar(255) NOT NULL,
   FirstName varchar(255),
   Age int,
);
```

Naming of a PRIMARY KEY constraint

- The following SQL creates a PRIMARY KEY on the "ID" column when the "Persons" table is created:

```
CREATE TABLE Persons (
    ID int NOT NULL,
    LastName varchar(255) NOT NULL,
    FirstName varchar(255),
    Age int,
    CONSTRAINT PK_Person PRIMARY KEY (ID,LastName)
);
```

Note: In the example above there is only ONE PRIMARY KEY (PK_Person). However, the VALUE of the primary key is made up of TWO COLUMNS (ID + LastName).

Naming of a PRIMARY KEY constraint

- PRIMARY KEY on ALTER TABLE

  To create a PRIMARY KEY constraint on the "ID" column when the table is already created, use the following SQL:

  ALTER TABLE Persons
  ADD CONSTRAINT PK_Person PRIMARY KEY (ID, LastName);

DROP a PRIMARY KEY Constraint

- To drop a PRIMARY KEY constraint, use the following SQL:

- ```
  ALTER TABLE Persons
  DROP PRIMARY KEY;
  ```

- ALTER TABLE Persons
  DROP CONSTRAINT PK_Person;
  ```

MySQL FOREIGN KEY Constraint

- MySQL FOREIGN KEY Constraint
- The FOREIGN KEY constraint is used to prevent actions that would destroy links between tables.
- A FOREIGN KEY is a field (or collection of fields) in one table, that refers to the PRIMARY KEY in another table.

MySQL FOREIGN KEY Constraint

- The table with the foreign key is called the child table, and the table with the primary key is called the referenced or parent table.
- Look at the following two tables:

PersonID	LastName	FirstName	Age
1	Hansen	Ola	30
2	Svendson	Tove	23
3	Pettersen	Kari	20

Parent – referenced table

Child table

OrderID	OrderNumber	PersonID
1	77895	3
2	44678	3
3	22456	2
4	24562	1

MySQL FOREIGN KEY Constraint

- The FOREIGN KEY constraint prevents invalid data from being inserted into the foreign key column, because it has to be one of the values contained in the parent table.

FOREIGN KEY on CREATE TABLE

The following SQL creates a FOREIGN KEY on the "PersonID" column when the "Orders" table is created:

```sql
CREATE TABLE Orders (
    OrderID int NOT NULL,
    OrderNumber int NOT NULL,
    PersonID int,
    PRIMARY KEY (OrderID),
    CONSTRAINT FK_PersonOrder FOREIGN KEY (PersonID)
    REFERENCES Persons(PersonID)
);
```

MySQL FOREIGN KEY Constraint

- The FOREIGN KEY constraint prevents invalid data from being inserted into the foreign key column, because it has to be one of the values contained in the parent table.

FOREIGN KEY on ALTER TABLE

To create a FOREIGN KEY constraint on the "PersonID" column when the "Orders" table is already created, use the following SQL:

```
ALTER TABLE Orders
ADD CONSTRAINT FK_PersonOrder
FOREIGN KEY (PersonID) REFERENCES Persons(PersonID);
```

To drop a FOREIGN KEY constraint, use the following SQL:

```
ALTER TABLE Orders
DROP FOREIGN KEY FK_PersonOrder;
```

MySQL AUTO INCREMENT Field

- What is an AUTO INCREMENT Field?
- Auto-increment allows a unique number to be generated automatically when a new record is inserted into a table.

- Often this is the primary key field that we would like to be created automatically every time a new record is inserted.

```
CREATE TABLE Persons (
    Personid int NOT NULL AUTO_INCREMENT,
    LastName varchar(255) NOT NULL,
    FirstName varchar(255),
    Age int,
    PRIMARY KEY (Personid)
);
```

Relational Database Management Systems (RDBMS).

“Relations" generally refer to the associations or connections between tables. These associations are established through keys, typically primary and foreign keys, and they form the basis of relational database management systems (RDBMS).

1. One-to-One Relationship:
2. One-to-Many Relationship:
3. Many-to-Many Relationship:

Relational database management systems (RDBMS).

- **One-to-One Relationship:**

In a hospital database, consider a Patients table and a MedicalHistory table. Each patient has exactly one medical history, and each medical history record belongs to only one patient.

Patients:
- PatientID (Primary Key)
- Name
- Age
- Gender

MedicalHistory:
- MedicalHistoryID (Primary Key)
- PatientID (Foreign Key)
- Diagnosis
- Treatment
- Date

- **One-to-Many Relationship:**

Consider a Departments table and an Employees table in a company database. Each department can have multiple employees, but each employee belongs to exactly one department.

Departments:
- DepartmentID (Primary Key)
- Name
- Location

Employees:
- EmployeeID (Primary Key)
- Name
- DepartmentID (Foreign Key)
- Position
- Salary

- **Many-to-Many Relationship:**

Imagine a Students table and a Courses table in a university database. Each student can enroll in multiple courses, and each course can have multiple students.

Students:
- StudentID (Primary Key)
- Name
- Age
- Gender

Enrollments:
- EnrollmentID (Primary Key)
- StudentID (Foreign Key)
- CourseID (Foreign Key)
- Grade

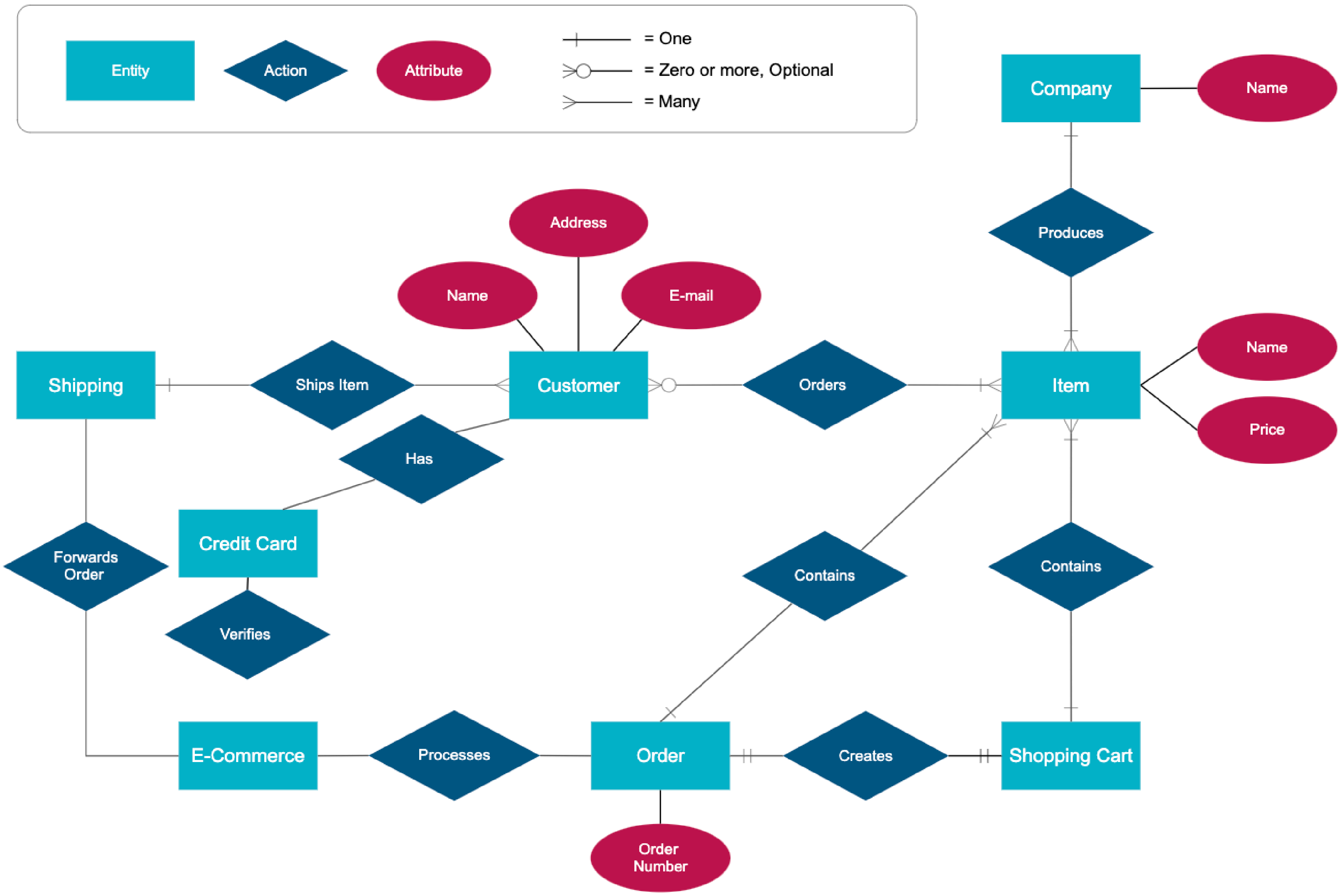
Pivot – bridge

Courses:
- CourseID (Primary Key)
- Title
- Description

Entity Relationship Diagram (ERD)

- An ERD visualizes the relationships between entities like people, things, or concepts in a database. An ERD will also often visualize the attributes of these entities.
- By defining the entities, their attributes, and showing the relationships between them, an ER diagram can illustrate the logical structure of databases.

Entity Relationship Diagram - Internet Sales Model

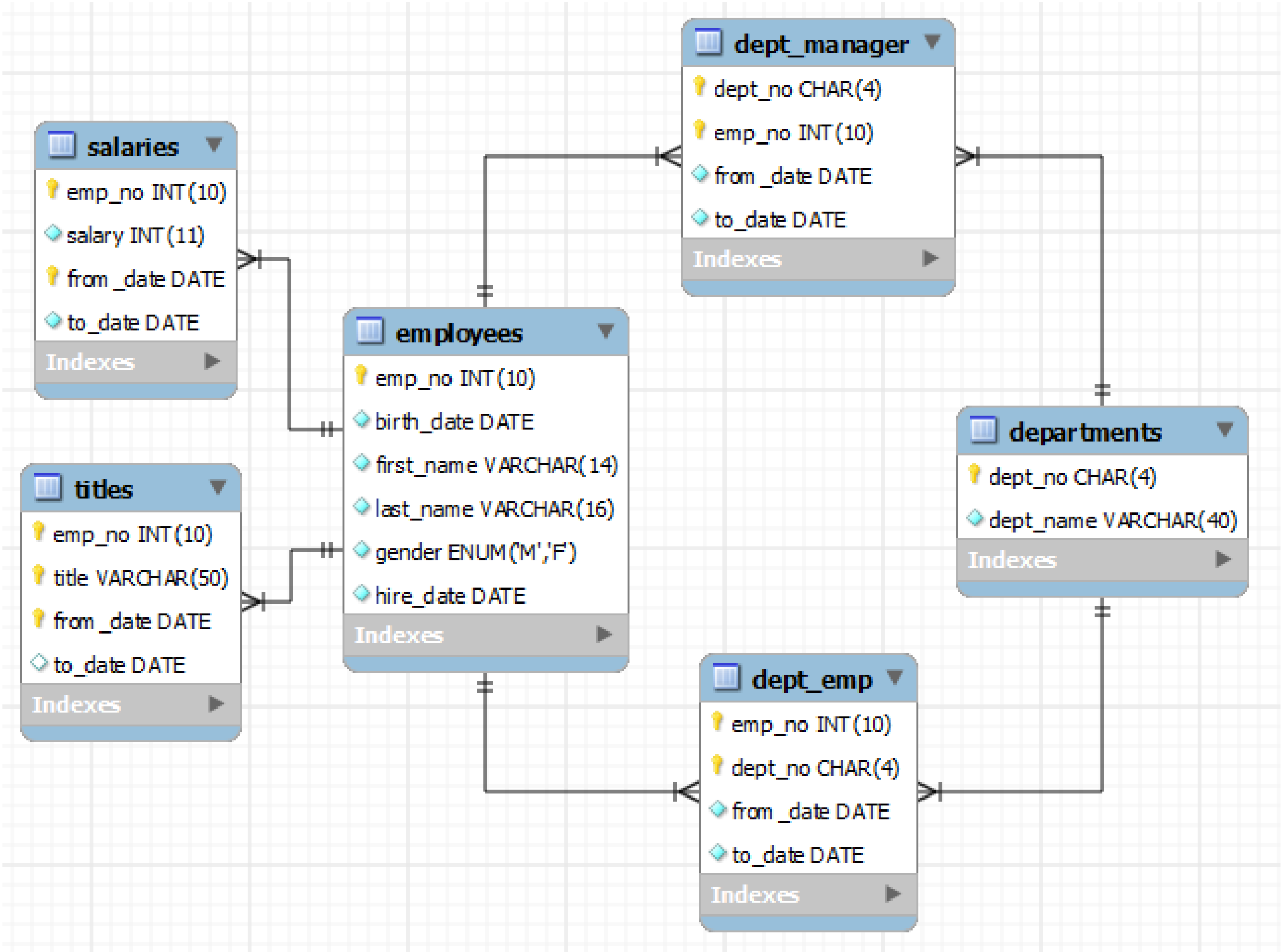


Entity Relationship Diagram (ERD)

EER (Enhanced Entity-Relationship) Diagram:

- Extends ERD by adding more complex relationships and features.
- Suitable for modeling more complex database structures.

EER (Enhanced Entity-Relationship) Diagram:



The following diagram provides an overview of the structure of the Employees sample database.

[Reference]

Tools to illustrate ERD

- Draw io
- MySQL workbench

Resources

- https://www.w3schools.com/